



亞洲大學 Asia University
資訊工程學系
Computer Science & Information Engineering



Midterm Project: Scrappy for scraping

Group: №12

Professor: /DINH-TRUNG VU (Assistant Professor) /

Student: /Ganbayar Gan-Erdene (113021202) /

Chapter 1: Introduction

1.1 GitHub

1. **Personal GitHub Account:** My personal GitHub account is:
<https://github.com/113021202>
2. **Group Project Repository:** Our group project repository is hosted on GitHub at:
<https://github.com/Advanced-CP/113021202> This repository includes files such as `readme.txt`, `LICENSE`, `113021202report.pdf`, and `see.png`, and represents our collaborative coursework.

1.2 Overview

This project uses **Scrapy**, a powerful Python-based web scraping library, to extract structured data from our group's GitHub repository. Scrapy is used to gather metadata including the repository URL, project description (About), the programming language used, number of commits, and the last update date.

Advanced Python features and libraries used include:

- `scrapy.Spider` class for structured crawling
- CSS selectors to parse HTML elements
- Built-in Python string manipulation methods

The spider successfully extracted the required repository metadata and exported it into an XML file format. This information can be used for project documentation, tracking contributions, and showcasing technical metadata.

Chapter 2: Implementation

2.1 Class 1: GithubSpider (scrapy.Spider)

This class is the main spider responsible for crawling and scraping the GitHub project page.

2.1.1 Fields:

- `name`: Identifier for the spider ('github_info')
- `start_urls`: List containing the GitHub repository URL to crawl

2.1.2 Methods:

- `parse(self, response)`: This is the main parsing method that is automatically called to extract data from the response received from GitHub.

2.1.3 Functions:

- Uses `response.css()` to extract:
 - Repository URL
 - About text using `p.f4.my-3::text`
 - Programming Language using `[itemprop="programmingLanguage"]::text`
 - Number of commits using `li.commits span.d-none.d-sm-inline strong::text`
 - Last updated time using `relative-time::attr(datetime)`

Chapter 3: Results

3.1 Result 1: Metadata Extraction

- Successfully extracted data fields:
 - Repository URL
 - About text (project description)
 - Language: Python
 - Number of commits
 - Last updated date

3.2 Result 2: XML Output

- The extracted data was saved into a file named `files.xml` in the XML format. It includes all the above metadata inside `<item>` tags.

Chapter 4: Conclusions

The use of Scrapy in this project enabled automated and structured data extraction from a GitHub repository. It demonstrated how web scraping techniques can be combined with Python's capabilities to collect metadata for software engineering projects. This tool can be extended for future projects that require dynamic extraction and analysis of online repository data.